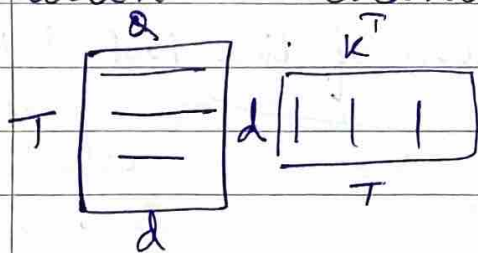


Week-11 LLM

L1

Motivation for fast attention mechanism, time & space complexity of attention

- models we have discussed so far, the context len(T) is 512 & 1024 $\rightarrow$ (GPT-2). But what if want to summarize or generate beyond T ?
- The seq. of an audio of 10s is abt. 80000, (sampling rate 8kHz). So model should cater to such longer sequences.
- Recently, the model's context len is increasing. One of the 1^o factor is the computational complexity of the self attention mechanism.



Size of emb. = $d = d_q = d_k$
context size = T

$d_q \cdot d_k^T \Rightarrow d$ multiplication & $(d-1)$ addⁿ.

$$\begin{aligned} (T \times d) \cdot (d \times T) &= T \times T \\ &\Rightarrow \boxed{T^2 \cdot d} \Rightarrow \text{time complexity} \\ \text{no. of tokens } T &\gg \text{embedding dim. } d. \end{aligned}$$

- what abt. ~~normalized~~ matrix?

//_

$$T \begin{bmatrix} QK^T \\ T \end{bmatrix} \Rightarrow A = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right)$$

for normalize each row $\Rightarrow O(T)$
 for entire $\Rightarrow O(T^2)$

then

$$T \begin{bmatrix} A \\ T \end{bmatrix} \times T \begin{bmatrix} V \\ d \end{bmatrix} \Rightarrow (T \times T) \times (T \times d)$$

$$\Rightarrow T^2 \cdot d$$

$$T^2 \cdot d + T^2 + T^2 \cdot d \Rightarrow O(T^2 d)$$

it is quadratic in T .

- Memory requirements

$B \rightarrow$ batch size

$T \rightarrow$ context len

$V \rightarrow$ vocab tokens

$d_{ff} \rightarrow$ feed forward

$$d_{model} = \frac{d_{ff}}{4}$$

$N \Rightarrow$ no. of params.

$n_h \Rightarrow$ no. of attenⁿ heads

for 1 sample

$$\begin{aligned} mha &= (Q_{T \times d} + K_{d \times T}^T + V_{T \times d}) n_h \\ &= 3 n_h (T \times d) \\ &= 3T \times d_{model} \quad (n_h d = d_{model}) \end{aligned}$$

batch $B \Rightarrow 3B T d_{\text{model}}$

$$Z = QK^T \Rightarrow B n_h T^2$$

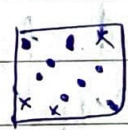
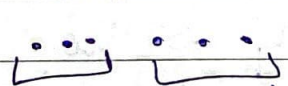
$$A = \text{softmax} \Rightarrow \boxed{B n_h T^2}$$

$\hookrightarrow$ quadratic w.r.t T
& linear w.r.t B .

- so how do we reduce the complexity for very large sizes of T ?

L2

Fast-Attenⁿ Mechanisms

- $O(T^2 d) \rightarrow$ we want to reduce it.
 $\hookrightarrow c T d \quad (c \ll T)$
- the neighbouring inner products are very impt. in self-attenⁿ & not all heads attend to all tokens.
- Strided local attenⁿ $\rightarrow$ attenⁿ wts. only for neighbours.  $\Rightarrow (3 T d)$
- Random local attenⁿ $\rightarrow$ "give attenⁿ to nearby, sometimes far ones also."
- Sparse block attenⁿ $\rightarrow$ , create blocks, within block they attend to every one of them.
- Global + local $\rightarrow$ first local, one could go for global.
- Flash $\rightarrow$ hardware-aware attenⁿ

L3

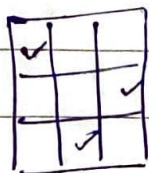
Local Attenⁿ Mechanisms

- full attenⁿ $\rightarrow O(T^2 d)$
 $O(T^2)$
- strided local attenⁿ $\rightarrow cTd$
 $\hookrightarrow$ every word attends to a window of c words ($\lfloor \frac{c}{2} \rfloor$ to the left & $\lfloor \frac{c}{2} \rfloor$ words to the right).
- Dilated attenⁿ - diff. dilaⁿ rates by diff. attenⁿ heads. No Δ in complexity.
- global + local $-(3T+T)d$. We have to manually set token that req. global attenⁿ based on the task.
- global + local + random $\rightarrow (3T+T+T)d$.
The BIGBIRD model proposes adding random tokens. Token like CLS req. full attenⁿ to get good performance on downstream tasks.
- Local block attenⁿ $\rightarrow O(\frac{T^2}{2}) \Rightarrow$ if $\geq$ blocks.

L4

Sparse Block Attenⁿ

QK^T $\rightarrow$ we can divide Q & K matrices into n blocks each. This will result in n^2 blocks in QK^T .



we want to select only 3 blocks.

//_

so whatever is the no. of blocks $\rightarrow n \Rightarrow \left\lfloor \frac{T^2}{n} \right\rfloor$

- $\pi = \text{perm}(0, 1, 2, \dots, n-1)$
let k^{th} element of π be $\pi(k)$. M matrix of size $T \times T$.

$$M_{ij} = \begin{cases} 1 & \text{if } \pi\left(\left\lfloor \frac{in}{T} \right\rfloor\right) = \left\lfloor \frac{jn}{T} \right\rfloor \\ 0 & \text{otherwise} \end{cases}$$

Block attenⁿ,

$$Z = \text{softmax}(QK^T \odot M)$$

for eg: $T=15$, $n=3$, then we have $3!$ ways of permuting the blocks.

$$Q = (Q_0, Q_1, Q_2) \in \mathbb{R}^{5 \times d}$$

$$K = (K_{\pi(0)}, K_{\pi(1)}, K_{\pi(2)}) \in \mathbb{R}^{5 \times d}$$

$$V = (V_{\pi(0)}, V_{\pi(1)}, V_{\pi(2)}) \in \mathbb{R}^{5 \times d}$$

$$\Rightarrow O\left(\frac{T^2 d}{n}\right)$$

apply same concept to MHA by allowing each head to use a different masking head.

- Blockwise sparsity captures both local & long distance dependencies in a memory efficient way using diff. permuteⁿ.
- It reduces the training-time reducⁿ significantly.
- But, perplexity score $\uparrow$, as $n \uparrow$.

L5 Low Rank Approximaⁿ

- Self-attenⁿ is low rank. So far, we are now here at $CTd \Rightarrow Td$. (But it does not capture the context.)
- RoBERTa model $\Rightarrow A = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right)$

apply SVD on a matrix A . Plot the normalized cumulative singular value averaged over 10k sentences. They got a long tail distribuⁿ. Hence, they said that A matrix is a low-rank matrix.

$$Q, K, V \in R^{T \times d}$$

introduce 2 learnable linear projecⁿ matrices

$$E, F \in R^{K \times T}$$

$$A = \text{softmax} \left(\frac{Q(EK)^T}{\sqrt{d}} \right)$$

$R^{T \times K}$ (pointing to Q)

$$D = AFV$$

$$\hookrightarrow T \times d$$

Now, time complexity is KTd .

L6 Fast Inference Mechanisms

- Naive implementaⁿ of autoregressive inference

I like

$h_0, \uparrow$	$h_0, h_1, \uparrow$	$h_0, h_1, h_2, \uparrow$
$\langle s, \text{start} \rangle$	$\langle s \rangle, I$	$\langle s \rangle, I, \text{like}$
$t = 0$	$t = 1$	$t = 2$

- at each time step, we compute the keys-values for all the previous queries as well. Waste of compute & increases the latency.
- KV caching $\rightarrow$ In 1st time step, compute the key-value pair. Store key-value in cache (in GPU RAM $\rightarrow$ HBM). For q_2 , reload k_1, v_1 from HBM & compute k_2, v_2 only.
- In this way, we trade off the memory for compute & compute scales linearly $O(n)$. We can't let it grow indefinitely.
- KV cache per token (in B) = $2 \times \text{num layers} \times (\text{num head} \times \text{dim head}) \times \text{precision in bytes}$
 in GPT3 $\rightarrow 9.4 \text{ MB (per token)}$
 2k context len $\rightarrow 19.3 \text{ GB}$
- MQA (multi-query attn) - same KV pair are shared across the heads.
 $\text{num heads} = 1$
 Greatly reduces the cache memory. Perform. greatly reduces. This req. to re-train the model with at least 50% of training data.
- GQA (grouped) - where a group of queries share key-value pairs that result in performance close to MHA.
- PA (paged) - fixed & conti. memory is reserved for KV caching. Len. of seq. is not known. Use a block of non-contiguous memory locaⁿ.